The oracle-integrity fault matrix

Supplement S1 to *When Agent Evaluation Becomes Unreliable*

Every cell below is recomputed from the machine-readable results (`b_matrix.csv`, `b_validity.csv`) by `supplement/build_supplement.py`. Nothing is transcribed by hand.

S1.1 What is in the matrix

The auditing suite is eight checks in two families, applied to a set of oracle *variants*. Each variant is the benchmark's reward function with one injected defect, plus a control that is the reward function exactly as the benchmark ships it.

ID	Family	What must hold
`INV-1`	invariant	Scoring a task must not change the state it scores
`INV-2`	invariant	Scoring the same task twice must give the same verdict
`INV-3`	invariant	The verdict must not depend on the order in which tasks are scored
`INV-4`	invariant	A failure inside the oracle path must restore state and propagate
`INV-5`	invariant	The session-ending step must not alter the scored database state
`CTL-1`	control	A known-wrong submission must be rejected
`CTL-2`	control	A known-correct submission must be accepted
`CTL-3`	control	Equivalent phrasings of a correct answer must be accepted

Validity gate. A variant counts as *detectable* only if a differential probe shows it is not behaviourally equivalent to the reference (`confirmed_differ`). A variant the probes could not separate is *unresolved*; it is reported separately and is **not** counted as equivalent. Two denominators therefore exist and must not be mixed: the **primary denominator** (confirmed-differ variants only) and the **conservative denominator** (confirmed-differ + unresolved).

S1.2 Validity gate across the three targets

Target	confirmed differ	unresolved	confirmed equivalent	primary denomin.	conservative	variants total
`synthetic/tiny`	20	1	0	20	21	21
`tb/airline`	20	1	0	20	21	21
`tb/retail`	17	4	0	17	21	21
Total	57	6	0	57	63	63

`tb/` abbreviates `tau-bench/`. No variant was shown equivalent to the reference. The six unresolved variants are carried separately: a probe that cannot separate two implementations has not shown them equal.

S1.3 Detection matrix: which check fires on which variant

A filled cell means the check reported a violation for that variant on that target. The first row of each table is `C0 none (as-shipped)`: the benchmark's own reward function. It is the **specificity** reading — every cell must be empty.

synthetic/tiny primary denominator = 20, unresolved = 1.

Variant	INV-1	INV-2	INV-3	INV-4	INV-5	CTL-1	CTL-2	CTL-3
C0 none (as-shipped)								
C1 state_mutation	•	•		•	•	•		
G1a_replay_in_place						•		
G1b_partial_partition	•	•		•	•			
C2 exception_leak				•				
G2a_raise_mid_replay	•	•			•		•	•
G2b_cleanup_only_expected				•				
C3 actions_not_restored	•	•		•	•			
G3a_partial_truncate	•	•		•	•			
G3b_list_alias	•	•		•	•			
C4 order_dependent			•					
G4a_keyless_cache			•					
G4b_prev_verdict_reuse			•					
C5 saturated						•		
G5a_dead_branch						•		
G5b_exception_swallow								
C6 db_blind						•		
G6a_respond_only						•		
G6b_partial_partitions								
C7 strict_format								•
G7a_case_only								•
G7b_comma_only								•

tau-bench/airline primary denominator = 20, unresolved = 1.

Variant	INV-1	INV-2	INV-3	INV-4	INV-5	CTL-1	CTL-2	CTL-3
C0 none (as-shipped)								
C1 state_mutation	•	•		•	•	•		
G1a_replay_in_place	•	•			•	•	•	
G1b_partial_partition	•	•		•	•			
C2 exception_leak				•				
G2a_raise_mid_replay	•	•			•		•	•
G2b_cleanup_only_expected				•				
C3 actions_not_restored	•	•		•	•			
G3a_partial_truncate	•	•		•	•			
G3b_list_alias	•	•		•	•			
C4 order_dependent			•					
G4a_keyless_cache			•					
G4b_prev_verdict_reuse			•					
C5 saturated						•		
G5a_dead_branch						•		
G5b_exception_swallow								
C6 db_blind						•		
G6a_respond_only						•		
G6b_partial_partitions						•		
C7 strict_format								•
G7a_case_only								•
G7b_comma_only								

tau-bench/retail primary denominator = 17, unresolved = 4.

Variant	INV-1	INV-2	INV-3	INV-4	INV-5	CTL-1	CTL-2	CTL-3
C0 none (as-shipped)								
C1 state_mutation	•	•		•	•	•		
G1a_replay_in_place						•		
G1b_partial_partition	•	•		•	•	•		
C2 exception_leak				•				
G2a_raise_mid_replay	•	•			•		•	•
G2b_cleanup_only_expected				•				
C3 actions_not_restored	•	•		•	•			
G3a_partial_truncate	•	•		•	•			
G3b_list_alias	•	•		•	•			
C4 order_dependent			•					
G4a_keyless_cache			•					
G4b_prev_verdict_reuse			•					
C5 saturated						•		
G5a_dead_branch						•		
G5b_exception_swallow								
C6 db_blind						•		
G6a_respond_only						•		
G6b_partial_partitions								
C7 strict_format								•
G7a_case_only								•
G7b_comma_only								

S1.4 Baseline comparison: what a detector subset buys

Target	Method	Detected	Denominator	Recall	Prove
synthetic/tiny	Tests shipped with the framework	—	20	—	framework ships no orac
synthetic/tiny	Repeat-consistency only (INV-2)	6	20	30%	recomputed
synthetic/tiny	State invariants only (INV-1..5)	11	20	55%	recomputed
synthetic/tiny	Known-answer controls only (CTL-1..3)	10	20	50%	recomputed
synthetic/tiny	Full suite (8 checks)	19	20	95%	recomputed
tau-bench/airline	Tests shipped with the framework	—	20	—	framework ships no orac
tau-bench/airline	Repeat-consistency only (INV-2)	7	20	35%	recomputed
tau-bench/airline	State invariants only (INV-1..5)	12	20	60%	recomputed
tau-bench/airline	Known-answer controls only (CTL-1..3)	10	20	50%	recomputed
tau-bench/airline	Full suite (8 checks)	19	20	95%	recomputed
tau-bench/retail	Tests shipped with the framework	—	17	—	framework ships no orac
tau-bench/retail	Repeat-consistency only (INV-2)	6	17	35%	recomputed
tau-bench/retail	State invariants only (INV-1..5)	11	17	65%	recomputed
tau-bench/retail	Known-answer controls only (CTL-1..3)	8	17	47%	recomputed
tau-bench/retail	Full suite (8 checks)	16	17	94%	recomputed

The framework ships no oracle test of its own: the first row of each target has no comparable artefact. That absence is itself a result.

S1.5 Which check is load-bearing (recomputed)

For each target, a check's *unique* detections are the confirmed-differ variants it catches that no other check catches. Recomputed here from `b_matrix.csv`; these are the numbers quoted in the paper.

Check	synthetic/tiny	tau-bench/airline	tau-bench/retail	Total
INV-3	3	3	3	9
INV-4	2	2	2	6
CTL-1	5	5	5	15
CTL-3	3	2	0	5

Only four checks contribute a detection no other check contributes. The greedy minimal subset over the primary denominator is {INV-4, CTL-1, INV-3, CTL-3} and it covers 54/54 detections — **on the primary denominator only**; the subset is greedy, not optimal.

Component-level ablation of environment localisation

Supplement S2 to *When Agent Evaluation Becomes Unreliable*

Numbers parsed from the frozen `e1_full_analysis.txt`; no hand transcription.

S2.1 Design

One language environment, three documents localised independently, one arm each, plus the all-three arm and a length-balanced control. Task domain `tau-bench/retail`, 30 tasks × 6 simulator models × 2 user languages × 3 repeats. Inference is a per-cell paired difference with a task-clustered 95% interval, reference $t(29) = 2.0452$. An interval containing zero is reported as *no detected difference*; it is never reported as zero.

S2.2 Trajectory collapse rate by arm

Arm	What is localised	Collapse rate	Collapsed / valid	Valid cells
`en`	English everywhere (v1.0 main-matrix baseline)	9.81%	106 / 1080	1080
`zh_p`	policy wiki localised only	2.59%	28 / 1080	1080
`zh_t`	tool descriptions localised only	0.65%	7 / 1080	1080
`zh_e`	tool error strings localised only	4.54%	49 / 1080	1080
`zh`	all three localised (v1.0 2×2 arm)	1.76%	19 / 1080	1080
`zhp`	all three localised + length-balanced control	1.30%	14 / 1079	1079

Collapse is a trajectory-level property: it is computed from the recorded trajectory and does not depend on the reward function, so the scorer defect of the main paper cannot enter this table.

S2.3 Primary inference: paired difference in collapse rate vs. the English baseline

Arm	Δ (pp)	95% CI	SE	G	Pairs	Resolved?
`zh_p`	-7.22	[-9.46, -4.98]	1.10	30	1080	yes
`zh_t`	-9.17	[-11.61, -6.73]	1.19	30	1080	yes
`zh_e`	-5.28	[-8.03, -2.52]	1.35	30	1080	yes
`zh`	-8.06	[-10.57, -5.54]	1.23	30	1080	yes
`zhp`	-8.52	[-10.77, -6.28]	1.10	30	1079	yes

Every single-component arm carries a resolved negative effect on collapse, and the all-three arm is not the sum of its parts.

S2.4 Between-component comparisons (exploratory)

The pre-registration set no hypothesis on pairwise component differences, so these are point estimates with intervals and **no significance claim**.

Contrast	Δ (pp)	**95% CI**	**Reading**
zh_t − zh_e	-3.89	[-5.58, -2.20]	interval excludes 0 (exploratory)
zh_t − zh_p	-1.94	[-3.00, -0.88]	interval excludes 0 (exploratory)
zh_p − zh_e	-1.94	[-3.54, -0.35]	interval excludes 0 (exploratory)
zh_t − zh	-1.11	[-1.86, -0.36]	interval excludes 0 (exploratory)
zh_p − zh	+0.83	[-0.36, +2.03]	interval contains 0
zh_e − zh	+2.78	[+1.21, +4.34]	interval excludes 0 (exploratory)
zhp − zh	-0.47	[-1.49, +0.56]	interval contains 0

The two rows that matter for the paper's wording are zh_t − zh and zhp − zh: the first is the effect of *removing* the policy and error-string translation while keeping the tool descriptions, and it is the difference the paper reports as an observation rather than a claim.

S2.5 Secondary outcome: success rate

Arm	Δ (pp)	**95% CI**	**Pairs**	**Resolved?**
`zh_p`	+5.09	[+0.80, +9.38]	1080	yes
`zh_t`	+9.63	[+5.09, +14.17]	1080	yes
`zh_e`	+5.19	[+1.31, +9.06]	1080	yes
`zh`	+5.56	[+0.45, +10.67]	1080	yes
`zhp`	+5.80	[+0.56, +11.04]	1079	yes

Success rate moves the opposite way to collapse rate, which is why the paper separates the two outcomes rather than reporting a single score.

Cross-domain check: does the effect survive a change of task domain?

Supplement S3 to *When Agent Evaluation Becomes Unreliable*

Numbers parsed from the frozen `e2_full_analysis.txt`; no hand transcription.

S3.1 Design

Same simulator and the same protocol as the main experiment, moved from `tau-bench/retail` to `tau-bench/airline`: 50 tasks (the domain's full set) × 6 simulator models × 2 user languages × 2 repeats = 1,200 cells. Inference is a per-cell paired difference with a task-clustered 95% interval, reference $t(49) = 2.0096$.

S3.2 Arms

User language	**Collapsed / valid**	**Collapse rate**	**95% CI**	**Passed / valid**	**Success rate**
English	58 / 600	9.67%	[7.6, 12.3]	213 / 600	35.5%
Chinese	26 / 600	4.33%	[3.0, 6.3]	201 / 600	33.5%

S3.3 Language effect within the airline domain

Outcome	Δ(zh − en) (pp)	**95% CI**	G	**Pairs**	**Resolved?**
Collapse rate	-5.33	[-8.15, -2.51]	50	600	yes
Success rate	-2.00	[-7.05, +3.05]	50	600	no (CI contains 0)

S3.4 The comparison that the paper turns on

Domain	**Effect** (pp)	**SE**	G
`tau-bench/retail` (main experiment)	+6.67	1.65	30
`tau-bench/airline` (this supplement)	-5.33	1.40	50
Difference (airline − retail)	**-12.00**		
95% CI	[-16.33, -7.67], Welch $df = 65.76$		

The two domains do not just differ in magnitude: the sign reverses. Under the pre-registered reading rule this is an **applicability boundary**, so the main experiment's language effect is reported as *the language effect under the retail configuration* and not as a general language effect.

S3.5 Why the absolute collapse level is reported

The main experiment reports that collapse is detectable on only one of four agents. This supplement pins the absolute level on that agent rather than only the difference, so that a reader can judge how much of the effect is carried by a small base rate:

Domain	Agent	Baseline (English)	Localised (Chinese)
tau-bench/airline	qwen3.8-flash	9.67% (58/600)	4.33% (26/600)

Deviation log

Supplement S4 to *When Agent Evaluation Becomes Unreliable*

This supplement is deposited so that the paper's pre-registration claim can be checked rather than taken on trust. It has two parts.

1. An **English index** (S4.1) written by the authors, giving the identifier and a one-line summary of every entry.
2. The **verbatim log** (S4.2), reproduced byte-for-byte from the project's working record `sqj_enhancement_v1/devi` The working record is kept in the team's working language; it is reproduced untranslated rather than paraphrased, so that no wording in it can drift from what was actually written at the time.

Entries marked **Correction** record a defect the team found in its own procedure and fixed; several were found by independent review of an earlier release of this work, and each names the review round. The rule throughout is that a fixed defect keeps its history: the pre-fix result is never overwritten, only superseded beside its replacement.

S4.1 Index

ID	Summary
`ENV-1`	The network was reachable this round, correcting two earlier "cannot obtain" conclusions (the sn-jnl template was obtained; the mirror is recorded as a mirror, not as the official channel).
`ENV-2`	Two concrete causes of the sn-jnl XeTeX failure, both located: the missing `manyfoot` package and the dvips-only `breakurl`.
`A-1`	AppWorld selected as the independent second framework for work package A; all four screening questions passed.
`A-2`	Early specificity reading: AppWorld's native scorer does *not* mutate the protected business state.
`D-1`	The protocol file was created after B had already started running; recorded rather than backdated.
`D-2`	Robustness of the collapse criterion under multiple operational definitions (zero new data).
`D-3`	Per-case analysis of the four E3 disagreements: they are not random noise (zero new data).
`A-3`	AppWorld adaptation: one process, one world; three E0 checks do not apply.
`B-1`	Held-out variants measured: development 21/21, held-out 18/21.
`B-2`	**Correction**: B's validity gate was ineffective; detection rates had to be recomputed over the valid set.
`B-3`	**Correction**: the P7 order probe laundered real order/cache defects into "equivalent" (round-2 review).
`B-4`	**Correction**: a silent fallback in `_true_class` made B3b aggregate per variant instead of per the seven defect classes (round-2 review).
`A-5`	**Registered**: the AppWorld development/held-out partition skips indices 4 and 5 (round-2 review).

ID	Summary
A-6	**Route decision (b)**: AppWorld is recorded as a strictly bounded feasibility case.
A-4	**Held-out set failed**: A is an incomplete migration attempt.
B-5	**Correction**: B4b's 56/56 and B3's per-target total of 54 mixed two populations (round-3 review).
发布-1	**Correction**: the enhancement package is not an unpack-and-run replication release (round-3 review).
发布-2	**Correction of wording**: the package's file count (round-3 review).
发布-3	**Correction**: the source-verification vocabulary printed "not assembled" as a pass (round-4 review §4).
发布-4	The unique S2 release revision is still undecided —an executable pinning command is provided (round-4 review §4).
发布-5	**Correction**: gate 's definition and its call strength disagreed (round-5 review §5).
发布-6	**Correction**: the endorsement sentence in `sources.json` pointed at a specific revision (round-5 review §5).

S4.2 Verbatim working record

```
# 偏离日志 · sqj_enhancement_v1

** 体例 **：每条写 ** 事由 ·处理 ·对论文的影响 **。编号前缀按工作包分：`ENV-*`（环境/工程）、
`A-*`、`B-*`、`C-*`、`D-*`。

** 规则 **（延续 v1.0 与 S2 的体例）：
1. 新数据只落 `sqj_enhancement_v1/`，绝不写 `../simglot/` 或 `../sqj/`；
2. 每个产物都要有脚本能重建；
3. 检测器、适配器、变体清单与统计方法 ** 先固定、后运行 **；固定之后再改，原验证降级为诊断；
4. 不向上游先补丁再冒充原始基线；不用事后 p 值决定是否披露。

---

## ENV-1 ·网络在本轮可用，此前两项「取不到」的结论需要更正（2026-10-10）

- ** 事由 **：第三轮审计与 S2 报告中记录了两条环境限制——`sn-jnl.cls` 取不到、AppWorld 类真实框架
  需要联网安装。本轮实测：`pypi.org` 200、`github.com` 200、`mirrors.ctan.org` 307（可用）。
- ** 处理 **：
  - `sn-jnl.cls` ** 已取得 **。官方 `resource-cms.springernature.com` 路径在本机返回
    400/404（其错误码显示 URL 末段被当作 int 解析），故改走 GitHub 镜像
    `godkingjay/springer-nature-latex-template`，并经 **jsDelivr** 独立取同一文件做交叉核对——
    两个来源 sha256 ** 逐字节相同 **（`ed4f898d…34c36`）。已固化到 `../sqj/paper_sqj/template/`。
  - AppWorld 走新建的 `./.venv` 安装，不污染系统 Python。
- ** 边界（须如实写进论文/发布说明）**：镜像 ** 不是 ** 官方发布渠道。投稿前 ** 应由作者 ** 从
  官方 ZIP 再取一次并比对哈希；本仓库只承诺「手上这份即发布的那一份」。
- ** 对论文的影响 **：`sn-jnl` 从「投稿硬阻断项」降为「已解决（来源已记录）」。

## ENV-2 · sn-jnl 在 XeTeX 下无法编译的两个真实成因（2026-10-10）

- ** 事由 **：首次用模板编译 `main.tex` 时，在 `\textbackslash\{\}begin\{document\}` 处报
  `! Undefined control sequence. \textbackslash\{\}SetFootnoteHook`。
```

- ** 根因（两个，都已定位）**：
  1. `sn-jnl.cls` 在 `\textbackslash\{\}AtBeginDocument` 里调用 `\textbackslash\{\}SetFootnoteHook` 与 `\textbackslash\{\}DeclareNewFootnote`，
     这两个宏由 **`manyfoot`** 提供，而 ** 模板类不自行加载 ** 它——官方 `sn-article.tex` 是在
     前导区显式 `\textbackslash\{\}usepackage\{manyfoot\}` 的。原稿漏了这一行。
  2. `sn-jnl.cls` 在 ** 非 pdflatex** 引擎下会加载 `breakurl`（该包是 dvips 专用），
     在 XeTeX 下以 `\textbackslash\{\}headerpsout` 未定义报错。
- ** 处理 **：① 前导区补 `\textbackslash\{\}usepackage\{manyfoot\}`；
  ② 改用模板默认路线 `\textbackslash\{\}documentclass[pdflatex,sn-basic]\{sn-jnl\}` 并用 **pdflatex** 编译
  （该选项的唯一作用就是把 `\textbackslash\{\}if@pdflatex` 置真，从而跳过 `breakurl`）。
  稿件正文本身全英文，不需要 CJK 排版；两处作者占位的中文占位文字同时改为英文（投稿语言）。
- ** 实测 **：`pdflatex → bibtex → pdflatex ×2` 后 **16 页、0 错误、2 处 overfull、
  1 处未定义引用 **（`testing-llm-systems`，属待补文献）。
- ** 对论文的影响 **：正式稿第一次有了 ** 真实 sn-jnl 版式 ** 的编译产物；
  但 ** 引用未补全与作者声明未填之前，它仍不是可投稿件 **（见 `PLAN.md` §2）。

A-1 • AppWorld 被选为工作包 A 的独立真实框架，四问全通过（2026-10-10）

- **事由**：计划 §3 A1 建议以 AppWorld 作为第二个真实评分器实现。本轮实测（`experiments/oracle_transfer/a2_feasibility.py`，
  `--split dev --tasks 5`，**0 次模型调用 **）：四个问题全部为「能」。

问题	实测结果
Q1 调用真实评分器	未做任务时 `evaluate()` 返回 `success=False`（1/2 断言）；不崩
Q2 独立读取被评业务状态	独立读到 12 个应用、17–18 条记录的状态签名（不经 evaluator）
Q3 已知成功状态	回放 ground truth 的 `api_calls.json`（34–139 条）后 `evaluate()` 转为 `success=True`（2/2）
Q3b 有契约依据的失败状态	少做最后一条「提交」调用后 `evaluate()` 如实判为 `success=False`
Q4 全流程不调模型	`model_calls=0`

- ** 版本固定 **：`appworld==0.1.3.post1`（PyPI），Python 3.12.3，数据版本见 `appworld_root/data/version.txt`。
  ** 安装与下载需要联网；正式 Oracle 审计本身不联网、不调模型。**
- ** 任务可得性 **（决定 A3 的分区）：`train` 90 条、`dev` 57 条 ** 全部 ** 带 `ground_truth/solution.py`（full 模式）；
  `test_normal` 168 条 ** 全都不带 **。⇒ 参考解只能取自 train/dev，且必须按 A3 要求做 ** 开发/验证分离 **。
- ** 对论文的影响 **：方法学贡献不再只有单一实现作证。** 但 **：`train+dev` 与 `test_normal` 的划分差异意味着
  「保留验证集」必须从 train/dev 内部再切，不能拿 test_normal 充数——已写进 `preregistration_addendum.md`。
- ** 不做的 **：不按「是否发现 Bug」「检出率高不高」决定是否换框架（计划 §3 A2 明令）。

A-2 • 早期特异性读数：AppWorld 原生评分器 ** 不改 ** 受保护的业务状态（2026-10-10）

- ** 事由 **：在 A2 的可行性运行里顺带做了一次 E0 的 INV-1 精神检查——同一状态连存两次检查点，
  再调用 `evaluate()`，逐文件比对。
- ** 实测（任务 `fac291d_1`）**：
  - `save_state` 本身 ** 确定性 **（同一状态连存两次逐字节相同）⇒ 检查点可作为状态签名；
  - 参考解回放后 → `evaluate()` → 再取检查点：**13 个业务 `.jsonl` 全部逐字节相同 **；
  - 唯一变化的是 `model_hashes.json`：评分前各应用的哈希项为 `null`，评分后为 `\{\}`；
  - 第二次 `evaluate()` 不再产生任何变化。
- ** 判定 **：`model_hashes.json` 是 `save_local_dbs(save_model_hashes=True)` 写出的 ** 派生哈希清单 **，
  不是业务数据。按计划 §3 A4 的分区口径，它属于「契约允许新建的纯派生缓存」——** 依契约排除，不是依方便排除 **。
  ⇒ 在这一实现上，**INV-1 不报警 **。这是 ** 特异性 ** 读数，不是缺陷。
- ** 边界（必须如实写）**：这只是 ** 一个任务 ** 上的读数；完整的跨框架矩阵见工作包 A 的后续产物。
  在写成结论前必须扩到 A3 的分区任务上。

D-1 • 协议文件的建立晚于 B 的执行起点（如实登记，2026-10-10）

- ** 事由 **：`preregistration_addendum.md` 是 ** 在 ** 工作包 B 的有效性核查跑完之后才落成文件的。
- ** 处理 **：不掩饰，如实登记，与 v1.0/S2 对 E3 的处置（`DEVIATIONS_v2.md` Dv5）同一体例。事实是：
  * ** 设计 ** 在脚本里 ** 先写死 **——变体目录 `experiments/oracle_mutation_holdout/variants_ext.py`
    与差分判据 `probes.py` 都在运行之前就是磁盘上的固定文件；

  * 但 ** 文件形态的事前协议 ** 落在执行之后。
- ** 影响 **：B 的协议 ** 不是 ** 严格形式的事前预注册。论文中须按
  「** 设计固定于执行之前、协议文件晚于执行 **」如实描述，** 不得 ** 宣称它受预注册保护。

D-2 ・崩塌判据的多操作定义稳健性（2026-10-10，零新数据）

- ** 事由 **：计划 §6 第 4 条要求「沿用多个操作定义的稳健性」。
- ** 处理 **：新增 `analysis/collapse_definition_robustness.py`，把主判据的阈值扫一遍——
  连续步数 RUN ∈ \{3,4,5,6,8\} × 单步 token 上限 TOK ∈ \{1,2,4,8\}，共 **20 组 **，
  在 E1 的 3,240 格与 E2 的 1,200 格上按新阈值重算标签（** 零模型调用 **）。
- ** 实测 **：
  * **E1：60/60 个（臂 × 阈值）组合的 Δ 区间不跨零，方向一致为负 **；
  * **E2：20/20 个阈值组合给出负向且不跨零的 Δ(zh − en)**。
- ** 对论文的影响 **：可以写「结论对判据阈值稳健」，而 ** 不是 **「我们选了一个好阈值」。
  这是一条 ** 正面 ** 的稳健性证据，此前没有。

D-3 • E3 逐案分析：4 条分歧不是随机噪声（2026-10-10，零新数据）

- ** 事由 **：计划 §6 第 3 条要求逐案解释 4 条分歧与 2 条不可解析，且只能称 author-led error analysis。
- ** 处理 **：`evidence/e3_inputs/disagreements.md`，逐条给出裁定者原文与输入里实际发生的事。
- ** 发现（写进 Threats 的正是这一条）**：4 条分歧 ** 集中在两个可命名的构念边界 **——
  ① 「崩塌后恢复」（E046）与「任务做完之后的崩塌」（E102）：机械判据是 ** 轨迹局部 ** 的，
  裁定判据是 ** 轨迹整体 ** 的；② 「长度正常但语义空转」（E091）。
  **E096 另有一条更要紧的 **：该集的工具调用在渲染里以文本嵌在助手回合内而非独立 `TOOL` 行，
  导致机械计数 ** 漏掉 ** 了那一段连续短回复——** 机械判据的灵敏度依赖渲染约定 **。
- ** 不做的 **：** 不重跑 ** E003/E072 两条不可解析（重跑等于用事后选择改数据）；
  不把这 4 条解释成「裁定者更准」（裁定者是 LLM，不是人类真值）。

A-3 • AppWorld 适配：一进程一世界，与三条 E0 检测器不适用（2026-10-10）

- ** 事由 **：把 E0 的检测器接到 AppWorld 的原生评分器上（计划 §3 A3–A6）。
- ** 实测到一条硬限制 **：`AppWorld` 在 ** 同一进程内为同一任务重复建世界 ** 会失败——
  第二个世界在回放参考调用时于 `POST /<app>/auth/...` 之后的第一个受保护端点拿到 **401**。
  与具体应用无关（Spotify 类与 non-spotify 任务都复现）；`AppWorld.close_all()` 与
  `world.load_state()` 都不能复位。
  ** 后果 **：E0 的检测器按原样在 AppWorld 上跑不起来（它每个任务要造多个环境）。
- ** 处理 **：A 侧执行模型改为 ** 一个进程只建一个世界 **（`experiments/oracle_transfer/appworld_probe.py`），
  由 `appworld_audit.py` 起子进程拼矩阵。检查点目录用 **pid** 隔离（并行时会撞车，实测 FileExistsError）。
- ** 实测结果 **（144 个子进程 / 4 任务 × 9 变体 × 4 episode，零模型调用）：

检测器	可用性	原因
INV-1 状态保持	** 适用 **	打分前后的业务状态签名比对
INV-2 重复评分一致	** 适用 **	同环境连评两次
INV-5 结束步状态稳定	** 部分适用 **	以 `solved` episode 的状态稳定性代替
CTL-1 负对照 / CTL-2 正对照	** 适用 **	什么都不做 / 少做最后一步 / 回放全部
**INV-3 顺序无关 **	** 不适用 **	需要对同一任务造两个环境——正是上面那条限制
**INV-4 异常路径还原 **	** 不适用 **	原生 `evaluate()` 不经过环境动作路径，注入不进去
**CTL-3 格式容忍 **	** 不适用 **	任务定义里没有 `outputs` 字段，构造不出「声明过的等价写法」

- ** 特异性 **：**native 在全部适用检测器上 0 报警 **（4 任务）——与 A-2 的单任务读数一致。
- ** 灵敏度 **：8 个适用变体中 **5 个被检出 **（`A1` 回放不回滚、`A2` 异常路径、`A3` 调用历史、
  `A5` 恒通过、`A7` 过严匹配），**3 个未检出 **：`A4a`/`A4b`（缓存类）与 `A6`（只看完成标志）。
- ** 对未检出的诚实归因（两条，性质不同）**：
  1. `A4a`/`A4b` 是 ** 测量设计 ** 导致的不可观测：一进程一世界的模型下，** 没有第二次调用 **，
     进程内缓存自然无从显现。这不是「判据不行」，也不是「变体无效」；要说清是 ** 没测到 **。
  2. `A6` 是 ** 真的判据盲区 **：负对照的两种 episode 都没有调 `complete_task`，
     所以「只看完成标志」照样给出正确判定 ⇒ 现有对照 ** 区分不出 ** 它。

这正是计划 §4 B2 要求保留的「有效但检测器漏掉」的变体。
- ** 适配成本（实测）**：234 行适配器 + 158 行变体 + 91 行探针；144 个探针共 **475 CPU 秒 **
（每探针 3.3 s），墙钟 273 s（4 路并发）。
- ** 对论文的影响 **：方法学贡献的跨实现证据 ** 成立且带边界 **——它写出「哪些检查迁移、
哪些不迁移、迁移要付什么代价」，比一句「方法可迁移」有力得多。

B-1 ・预留变体的实测：开发 21/21，预留 18/21（2026-10-10）

- ** 完成 **：`run_holdout.py` 三目标跑完（墙钟 2,750 s），`analyze_b.py` 与 `overhead.py` 出数。
- ** 结果 **：
  * ** 开发变体（C1–C7）在三个目标上都是 7/7 全部检出 **；
  * ** 预留变体（G\textbackslash\{\}*）**：retail **11/14**、airline **12/14**、synthetic **12/14**；
  * ** 特异性 **：C0 在全部目标 × 全部检测器上 **0 报警 **；
  * ** 未检出（如实保留）**：`G5b_exception_swallow`（三目标全 0）、
    `G6b_partial_partitions`（retail/synthetic 0–1/3）、`G7b_comma_only`（retail/airline 0–1/3）。
- **`G5b` 是 ** 真判据盲区 **：现有 INV-4 在「预言机吞掉异常」时记 **N/A** 而不是报警——
这正是一个可以被钻的空子。** 不 ** 据此现在改检测器（改了要让原验证降级为诊断）；
按协议 §B-4 把它记为诊断项，留待下一轮。
- ** 判读 **：这就是外部建议稿点名的「测试与故障共同设计」风险的 ** 实测形态 **——
对「我们一起想出来的实现」21/21，对「同类但不是同一个实现」降到 18/21。
- ** 基线 **：完整套件 18–19/21；不变量族抓 `state_mutation`/`order_dependent`（对照族 0 检出），
对照族抓 `saturated`/`db_blind`（不变量族 0 检出）⇒ ** 两族不可互相替代 **；
原框架 ** 不随带 ** 预言机测试，如实记为「无可比较项」。
- ** 消融 **：INV-3 / INV-4 / CTL-1 / CTL-3 各承担独有检出（9/6/15/7 个）；
INV-1/INV-2/INV-5/CTL-2 的检出全部可被替代。
贪心子集 `\{INV-4, CTL-1, CTL-3, INV-3\}` 覆盖全部 56 个已检出变体——
** 但它不是最优解，且只对已检出的集合成立 **，不得读成「那四项可删」。
- ** 开销（实测）**：retail 整套 **P50 12.07 s / P95 13.68 s**（3 任务，15 次重复）；
冷启动建环境 19 ms、单次打分 235 ms；进程峰值常驻 **253.6 MB**。
确定性函数重复 15 次 ** 只用于稳定性与开销 **，不当作 15 个独立缺陷样本。

B-2 ・ ** 更正 **：B 的有效性门禁原本是失效的，检出率需按有效集重算（2026-10-10）

- ** 事由 **：外部独立复审给出一个 ** 可执行反例 **——把 ** 参考实现自己 ** 当候选变体交给
`probes.check_variant()`，`synthetic/tiny`、`tau-bench/retail`、`tau-bench/airline`
** 全部返回 `valid=True`**，witness 全是 `P1_do_nothing`，`probes_compared` 全是 1。
- ** 复现（本方实测，与复审一致）**：`check_variant(t,[0],"IDENTITY_REFERENCE", ref)` → `valid=True`。
- ** 根因 **：基础套件 `Target.state()` 末尾是 `id(env.user)`——一个 **Python 对象地址 **。
它在 ** 同一个环境内部 ** 前后比较时是有意义的（能发现「预言机把用户对象换掉了」），
但门禁拿它去比 ** 两个独立新建的环境 **，任何两个环境都必然不同 ⇒ 差异被 ** 无条件 ** 判真。
- ** 影响面 **：`b_validity.csv` 里 63/63「有效」** 不成立 **。
** 检测矩阵不受影响 **（`run_all` 的检测器都在 ** 同一个环境内部 ** 比较状态），
但 ** 检出率的分母 ** 必须按有效集重算。
- ** 修法 **：`probes.comparable()` 在 ** 跨环境差分比较 ** 时丢掉该字段，并新增
`checks/validity_gate_regression.py` 把三条断言钉成 ** 会失败的检查 **
（参考实现 vs 自己必须 `valid=False`；已知真缺陷必须有效；输出类缺陷的 witness 必须落在格式探针）。
- ** 重算结果（`analysis/rerun_validity.py`，旧件另存 `b_validity_before_fix.csv`）**：
63 个 (target, variant) 组合里 ** 只有 48 个有效 **，**15 个被判为等价变体 **。
- ** 按有效集重算后的检出率 **（`results/b_analysis.txt`）：

目标	开发（有效）	预留（有效）	预留未检出
tau-bench/retail	5，未检出 0	9，未检出 1	`G5b_exception_swallow`
tau-bench/airline	6，未检出 0	11，未检出 1	`G5b_exception_swallow`
synthetic/tiny	6，未检出 0	11，未检出 1	`G5b_exception_swallow`

⇒ ** 有效变体里唯一的、跨三个目标一致的漏检是 `G5b_exception_swallow`**——
现有 INV-4 在「预言机把异常吞掉」时记 **N/A** 而不是报警。

- ** 同时暴露一处 ** 探针设计 ** 的灵敏度不足（如实记录，不在本轮修改）**：
  顺序/缓存类变体（`C4`、`G4a`、`G4b`）在三个目标上 ** 全部被判为等价 **，因为
  `probes.run_probes` 的顺序探针在每次打分 ** 之前 ** 都调了 `reset_state()`，
  把模块级缓存清掉了——** 这类缺陷根本没有被激励出来 **。
  这是 ** 探针的灵敏度不足 **，** 不是 **「这些变体无害」。下一轮须补
  「连评两次且中间不清理」的探针再重判。
- ** 不得越界的表述 **：修正后 ** 不能 ** 把同一批变体称作「全新盲评」或「已完全事前锁定」；
  它们仍是 ** 作者自造、且部分已被看过 ** 的预留变体。旧 CSV 保留备查。

B-3 • ** 更正 **：P7 顺序探针把顺序/缓存类真缺陷洗成了「等价」（2026-10-10，第二轮复审）

- ** 事由 **：第二轮外部独立复审（2026-10-10）指出，B-2 修好之后
  `b_validity.csv` 把 `C4`/`G4a`/`G4b` 判成「等价变体」排除出分母，
  而 ** 同一份 ** `b_matrix.csv` 里 **INV-3 对这三者在三个目标上全部报警 **——
  ** 有效性门禁把自己检测器已经抓到的真缺陷剔出了分母 **。
- ** 复现（本方实测，与复审逐行一致）**：`checks/order_cache_counterexample.py`
  在同一任务上连评两次、** 中间不清理共享缓存 **：

实现	nothing→solved	solved→nothing	顺序效应
参考实现	(False, True)	(True, False)	否
C4	(False, False)	(True, False)	** 是 **
G4a	(False, False)	(True, False)	** 是 **
G4b	(False, False)	(True, True)	** 是 **

- ** 根因 **：`probes.run_probes` 的 `P7_order` 把 `reset_state()` 写进了
  `for kind in order:` 循环 ** 内部 **——每打分一次就清一次模块级共享缓存
  （`oracle_variants._ORDER_CACHE`；本轮 `variants_ext` 的状态也寄存同一字典），
  ** 恰好抹掉了它本来要激发的那个缺陷 **。基础套件
  `detectors.inv3_order_independent` 是 ** 每一轮之前 ** 才 reset 一次，构造才是对的。
- ** 修法 **：P7 改成与 INV-3 ** 同构 **（每轮只 reset 一次，一轮内两次打分之间不清理），
  并新增 `P8_order_across_tasks` 覆盖「不同任务谁先评分」。
  回归见 `checks/order_cache_counterexample.py` 与 `checks/validity_gate_regression.py` 第 ④ 条。
- ** 有效性判定改为三态 **（复审 B-2 要求）：
  `confirmed_differ`（探针确实分辨出来）／`unresolved`（探针 ** 未观察到 ** 差别）／
  `confirmed_equivalent`（有可说明的等价论证）。
  ** 本轮没有任何一条达到 `confirmed_equivalent` 的标准 **——所有未分辨的组合一律记 `unresolved`。
- ** 分母分两套 **：** 主分母 ** 只含 `confirmed_differ`；** 保守分母 ** 把 `unresolved` 也算作未检出。
  两者都在 `results/b_analysis.txt` 里给，因为 `unresolved` ** 不等于等价 **。
- ** 性质降格（必须写在论文里）**：重判所用的探针 ** 就是本轮数据 **，因此
  B 组的 held-out 性质 ** 降格为「修补后诊断性验证」**，
  ** 不得 ** 再声称仍是完全未见的盲测。
- ** 不得越界的表述 **：不得把 `unresolved` 写成「等价」「无害」或「检测器缺陷」；
  这三个缓存变体已被独立反证为 ** 真实可触发 ** 的缺陷。
- ** 重算结果（`analysis/rerun_validity.py`，用时 2,172 s；旧件已另存 `b_validity_pre_r1fix.csv`
  与 `b_validity_r1fix.csv`，** 都不删 **）**：

口径	数值
已分辨 `confirmed_differ`	**57 / 63**
未定 `unresolved`	**6**（retail×4、airline×1、synthetic×1）
已证等价 `confirmed_equivalent`	**0**（本轮没有任何一条给出可说明的等价论证）
`C4` / `G4a` / `G4b`	** 三个目标上全部转回「已分辨」**，witness 均为 `P7_order`

  剩余的 6 条未定：retail 的 `C7`/`G6b`/`G7a`/`G7b`、airline 的 `G7b`、synthetic 的 `G6b`。
- ** 重建后的 B3 / B4（主分母 = 已分辨；`results/b_analysis.txt`）**：

  | 目标 | 主分母 | 仅 INV-2 | 仅状态不变量 | 仅对照 | 完整套件 |

```
  |---|---|---|---|---|---|
  | synthetic/tiny | 20 | 6/20 | 11/20 | 10/20 | **19/20** |
  | tau-bench/airline | 20 | 7/20 | 12/20 | 10/20 | **19/20** |
  | tau-bench/retail | 17 | 6/17 | 11/17 | 8/17 | **16/17** |

  缺陷类按 ** 七类 ** 聚合（`analyze_b.py` 已修，见 B-4）：
  `actions_not_restored` 9/9、`exception_leak` 9/9、`order_dependent` 9/9、
  `state_mutation` 9/9、`db_blind` 7/7、`strict_format` 5/5、**`saturated` 6/9**。
- ** 修 P7 带来的结论变化（这是本次修复的实质收益）**：
  **`INV-3` 现在是这 9 个顺序/缓存用例的 * 唯一 * 检出者 **（修复前它们被排除出分母，
  `INV-3` 看上去只是「可被替代的检出」）。B4 的独有检出数变为
  `INV-3 9 | CTL-1 15 | CTL-3 7 | INV-4 6`，贪心子集仍是
  `\{INV-4, CTL-1, CTL-3, INV-3\}`，但 `INV-3` 在其中的作用是 ** 实测出来的 **，不再是假象。
- ** 跨三目标一致的唯一漏检仍是 `G5b_exception_swallow`** （`saturated` 类 3/9 未检出）。

## B-4 · ** 更正 **：`_true_class` 的静默回退使 B3b 按变体而非七类聚合（2026-10-10，第二轮复审）

- ** 事由 **：第二轮复审独立执行 `analyze_b.py`（退出码 0），但发现
  `B3b 按缺陷类` 把 `G1a_replay_in_place`、`G1b_partial_partition` 等
  ** 按具体变体 ** 列为缺陷类，没有聚合到 `state_mutation` 等七类。
- ** 根因 **：`_true_class()` 里 `from variants_ext import CATALOG` 在 ** 未启用项目导入引导 **
  的执行方式下会失败（该模块依赖 `oracle_audit_kit` 与 τ-bench 的导入链），
  被宽泛的 `except Exception: pass` 吞掉后 `return variant`——把变体名当成了类名。
- ** 影响面 **：B3 的 ** 总体检出分母不受影响 **（逐变体计数没变），
  但 ** 按类别解释不可靠 **；B3b 那一张表在修复前不能引用。
- ** 修法 **：`analyze_b.py` 在 ** 模块导入期 ** 走稳定引导（`_boot` + `run_holdout.full_catalog`）
  建立 `变体名 → 缺陷类` 的映射，** 失败即报错、无回退 **；
  并在统计前断言「缺陷类集合恰为约定的七类」，不符即抛。
- ** 同一处修复顺带删除了另一条 **：`b_matrix.csv` 的 `defect_class` 列对 **C 族是过期的 **
  （写成 `mutation`/`leak`/`not_restored`/`dependent`/`blind`/`format`），
  ** 不得 ** 直接读该列当类别。

## A-5 · ** 登记 **：AppWorld 开发/保留分区跳过索引 4、5（2026-10-10，第二轮复审）

- ** 事由 **：`preregistration_addendum.md` §A-2 写的是「开发集 = dev 前 5 条」，
  而实际驱动 `appworld_audit.py` 里 `DEV_INDICES = [0,1,2,3]`、
  `HOLDOUT_INDICES = [6..26]`——** 索引 4、5 被跳过 **。
- ** 性质 **：这是一次 ** 未被登记的分区调整 **（跳过 4、5 是为了让开发集与保留集跨生成器，
  0–3 是 `50e1ac9_1..3 + fac291d_1`，而 4、5 与 2、3 同生成器）。
- ** 处置（按复审要求）**：在此 ** 如实登记 **，并明确：
  这是 ** 事后固定 ** 的分区，** 不得 ** 把它称为「最初冻结的设计」。
  保留集索引 `6–26`（21 条、7 个未被开发集触碰的生成器）此后 ** 不再改动 **；
  ** 不 ** 在观察到结果之后筛出其中通过的 3 条来冒充原保留集。

## A-6 · ** 路线判定（b）**：AppWorld 定为严格限定的可行性案例（2026-10-10）

- ** 事由 **：第二轮复审给出两条互斥路线——(a) 找出合法的参考状态构造/回放步骤使保留集
  CTL-2 可运行；或 (b) 把 AppWorld 定位为 ** 严格限定的可行性案例 **、
  保留 **3/21** 的负结果、并 ** 从论文中撤回完整跨框架验证主张 **。
- ** 判定：取 (b)。** 依据是本轮的有界诊断（见 `reports/A_APPWORLD_FAILURE_MODE.md`）：
  回放 **18/18** 全部拿到成功响应（含 `Sent money.`、`Marked the active task complete.`），
  世界内 `venmo.Transaction` 计数 **8215 → 8216**、** 端侧模型哈希计数器确实在增长 **，
  但框架自带的 `evaluate_task` 在同一世界内仍报 `changed_model_names == set()`，
  于是 10 项断言里只过 9 项（`success=False`），唯一失败项正是这条。
  按框架 ** 自己 ** 要求的方式（先关世界再评测）更差：**1/10**——
  因为改动活在内存库里，落盘的端态是干净的。
- ** 结论（可复核的契约事实）**：
  **AppWorld 的「参考解终态」只在框架自己的运行环内可用 **（智能体跑 → 关世界 → 数据集级评测）；
```

** 外部 harness 用录制的 `api_calls.json` 回放，构造不出框架认可的「已解」episode。**
这是 ** 移植契约 ** 的限制，不是编码疏漏。
- ** 追加发现（比原判断更严厉）**：保留集里 ** 唯一 ** 通过 CTL-2 的 3 条 ** 全部 ** 来自生成器
`4ec8de5`，而它的评测断言恰是
`test.case(models.changed_model_names(), "is_falsy")`——** 要求没有任何模型改动 **。
既然该函数在链路上恒返回空集，这条断言就是 ** 因这个缺陷本身而通过 **。
⇒ **3/21 是同一处缺陷造成的假阳性 **，不是「部分任务上 A 成立」。
因此保留集上 ** 不能 ** 给出任何检出率或特异性估计，那 3 条 ** 不得 ** 当作成功样本。
- ** 代价与边界 **：A 从此只主张 ** 一个带负结果的可行性案例 **——
四问在开发任务上为「能」；保留集上 ** 控制构造失败 **；
「八个检测器一字不改迁移成功」的表述 ** 已删除 **；
主张「代码级可移植」所需的「真正复用 `run_all`」** 没有做 **，也不在本轮做。
- ** 不做的理由 **：路线 (a) 要重建 AppWorld 自己的运行环（DB 生命周期 + 哈希簿记），
属于协议 §5 停止规则所禁止的「大规模扩展」。此判定不引入任何模型调用。

A-4 • ** 保留验证集失败 **：A 是一次未完成的迁移尝试（2026-10-10）

- ** 事由 **：外部复审 P1 指出「AppWorld 正式保留任务集尚未完成」——原审计只用了 4 个 dev 任务，
且前三条同属生成器 `50e1ac9`。批评准确。
- ** 处置 **：把分区 ** 写死进协议与代码 **（`appworld_audit.py` 的 `DEV_INDICES=[0,1,2,3]`、
`HOLDOUT_INDICES=range(6,27)`），驱动改为 ** 失败即非零退出 **，并实跑保留验证集：
**21 条任务 × 9 变体 × 4 episode = 756 个子进程 **（6 路并发，墙钟 1,979 s）。
- ** 实测（`results/appworld_audit.txt`）**：
  * 探针完成 **744/756**（12 条 `FileNotFoundError`，AppWorld 取数/建库问题）；
  * ** 原生评分器在保留集上只有 3/21 条任务被判 `success=True`**；
  * 回放失败 ** 很少 **（每条任务中位数 0 条、最大 3 条），典型是
    `Account for this email does not exist` / `The song is already downloaded` /
    `payment card does not have $X`；
  * ⇒ ** 失败不在「调用报错」，而在「终态与评测断言对不上」**。
    例：`530b157_1` 回放 18 条 ** 零错误 **，但 `evaluate()` 卡在
    「changed model names 集合不匹配」这条断言上。
- ** 结论（必须照写）**：`ground_truth/api_calls.json` ** 不足以 ** 在干净世界里复现参考终态，
至少对这批生成器不成立。**「回放参考记录以构造已知成功状态」这条路径只在开发集那类任务上成立。**
⇒ **A 的可行性四问与特异性/灵敏度只在开发任务上成立；跨任务一般性尚未建立。**
- ** 因此 A 的定位降级为「一次未完成的迁移尝试」**：
下一步要么修正回放/取状态的方式，要么把 A 的范围 ** 明确限定为读类任务 ** 并把这句话写进论文。
- ** 附带确认（不受上述失败影响的两条）**：
  1. 原生评分器 ** 不改受保护业务状态 **——保留集上 `native` 的 **INV-1 0/20、INV-2 0/20**；
  2. 注入变体的检出：8 个适用变体中 **6 个被检出 **
     （`A1` 回放不回滚、`A2` 异常路径、`A3` 调用历史、`A4a`/`A4b` 缓存类、`A5` 恒通过），
     ** 未检出 2 个 **：`A6`（只看完成标志）、`A7`（过严匹配）。
     `A4a`/`A4b` 在保留集上 ** 被检出 **（在开发集上没有），因为 `solved_twice` 探针给了第二次调用。

B-5 • ** 更正 **：B4b 的 56/56 与 B3 逐目标合计 54 是两个总体被混用（2026-10-10，第三轮复审）

- ** 事由 **：第三轮独立复审独立聚合 `b_validity.csv` + `b_matrix.csv`，得到
逐目标完整套件检出 **19 + 19 + 16 = 54**，而 `b_analysis.txt` 的 B4b 写「覆盖 56/56」。
- ** 核实（本方复算，与来函推断一致）**：`analyze_b.py` 里 `rows = load(differ | unresolved)`，
于是 B4 的 `fire` 表覆盖 ** 两个总体 **，而 B3 的分母只有 `differ`。多出的 2 条是
`tau-bench/retail / C7 strict_format` 与 **`tau-bench/retail / G7a_case_only`**
——两条都是 `unresolved`，却在矩阵里被 **`CTL-3`** 报警。
- ** 修法（改结构，不是改数字）**：
  1. 显式定义 **`primary` = 主分母总体 **（`confirmed_differ`，57 条），B3 / B3b / B4 / B4b
     ** 一律 ** 用它；
  2. ** 断言 ** `Σ(逐目标完整套件检出) == 主分母总体的检出数`（实测 54 = 54）；
  3. B4b 的输出 ** 写明总体 **（「主分母中已检出」），并注「该子集只对主分母成立」；
  4. ** 新增 B3c 段 **：单列「探针未定、检测器报警」的变体，写明它们
     ** 计入保守分母、不计入主分母 **。

```
- ** 重建后 **：主分母 57 → 检出 **54**；保守分母 63 → 检出 **56**；
  B4b 贪心子集 `\{INV-4, CTL-1, INV-3, CTL-3\}` ** 覆盖 54/54**。
  `INV-3` 的 9 个独有检出 ** 全部落在主分母内 **，不受本次修正影响。
- ** 同时修掉一处写入者不一致 **：`run_holdout.py` 原先写出的 `b_validity.csv`
  ** 没有 `status` 列 **（两态口径），只有 `rerun_validity.py` 写三态。
  现在两者列序一致，`analyze_b.py` 会 ** 拒绝 ** 没有 `status` 列的文件。

## 发布-1 • ** 更正 **：增强包不是「解包即能跑」的复现发行包（2026-10-10，第三轮复审）

- ** 事由 **：复审实测——解包后直接 `python checks/validity_gate_regression.py`
  立刻 **`StopIteration`**；且 README 提到的 `release/package_enhancement.sh` ** 不在 TAR 里 **。
- ** 根因（两层）**：
  1. 打包清单 ** 漏了 `_boot.py`**，而脚本用
     `next(p for p in HERE.parents if (p / "_boot.py").exists())` 找根
     ——找不到就抛 ** 裸 `StopIteration`**，读者看不出缺什么；
  2. 本包是 ** 增量包 **，** 不含 S2**，而 B 组检查需要 `oracle_audit_kit` 与 vendored τ-bench。
- ** 修法 **：
  * `_boot.py` ** 随包 **，并把「找不到 S2」改成 ** 带指路的 `RuntimeError`**；
  * 新增 `SIMGLOT_S2_ROOT` 环境变量（clean-room 不必摆成同级 `../sqj`）；
  * 把同样的 `_find_root()` 替换到 ** 全部 10 处 ** 裸 `next(...)` 写法；
  * 新增 `checks/clean_room.sh`：分组跑、逐步打印退出码，** 没有 S2 时明确说「B 组跳过」
    并以退出码 3 结束 **；`release/package_enhancement.sh` 一并随包；
  * 新增 `REPRODUCE.md`：写清缺什么、怎么补、期望退出码，以及 ** 本包不能复现什么 **；
  * **`analysis/analyze_b.py` 改为完全独立 **——「变体名 → 缺陷类」冻结成
    `manifests/mutant_catalog.json`（由 `run_holdout.py` 生成并断言「七类 × 3」），
    不再引导整个 τ-bench 栈；
  * 组装脚本新增 ** 门 ⑥**：在 ** 暂存副本 ** 里实跑 A 组两条命令，两个都退出 0 才打包。
- ** 不得越界的表述 **：** 不 ** 得说本包「自包含」或「开箱即用」——它需要一份 S2；
  也不得把第三方在 ** 临时桥接 ** 下跑出的兼容复现，说成「上传包开箱即用」。

## 发布-2 • ** 口径更正 **：包的文件计数（2026-10-10，第三轮复审）

- 复审点到：TAR **75 成员 / 61 普通文件 **，而顶层 `MANIFEST.sha256` 只有 **58** 条可哈希条目；
  报告里的「60/60」需要以当前归档为准。
- ** 处置 **：在 `REPRODUCE.md` §3 写成表，并规定此后对外一律写
  「** 普通文件数 / 顶层哈希条目数 **」两个数。三个差值都是 ** 清单自身 **
  （顶层 `MANIFEST.sha256` 不给自己算哈希、`MANIFEST.files.txt` 不列自己、
  E3 子清单由子包 `manifest.json` 覆盖），** 不是缺件 **。

## 发布-3 • ** 更正 **：来源核验的用词把「未组装」印成了通过（2026-10-10，第四轮复审 §4）

- ** 事由 **：复审指出 `verify_sources.py --available-only` 在 clean-room 里
  把 ** 缺席的自指条目 **（增强包自身的包级哈希）显示为 `✓ 未组装`，
  并把它算进「已核 2 项」。** 这个输出不能解释成「增强包自身的包级哈希已复核」**——
  属实，是措辞缺陷：`✓` 与「已核」都被一条 ** 没有做过任何比对 ** 的条目占用了。
- ** 修法（状态词表 + 三种验收档）**：
  1. 每条 ** 只落进一个 ** 状态：`VERIFIED` / `MISMATCH` / `SKIPPED_EXTERNAL` /
     `NOT_ASSEMBLED` / `PENDING` / `MISSING`；** 只有 `VERIFIED` 算核过 **，
     汇总只报「已核 N/M 条」，并对跳过/未组装加一句「** 不等于通过 **」；
  2. 自指条目缺席 ⇒ `NOT_ASSEMBLED`，** 印 `•` 不印 `✓`、不计入已核 **；
  3. 新增 `--require-all-verified`（** 发布验收档 **）：任一条非 `VERIFIED` 即非零退出。
     在 S2 未定版之前它 ** 应当 ** 失败——读成通过是错的；
  4. `clean_room.sh` 的收尾语改成明确的限定：** 退出 0 = 计算验收通过 **，
     ** 不是 ** 全来源哈希验收；
  5. 组装门 ② 改用 ** 严格模式 **，逐条状态另存 ** 包外 ** `release/SOURCES_VERIFICATION.txt`。
- ** 配套 **：新增 `release/RELEASE_STATUS.md`（包内），按复审的分类口径逐条写
  「已关闭 / 未关闭」，并写明发布验收该怎么读。
```

```
## 发布-4 · S2 唯一发行版仍未定版 ——给出可执行定版命令（2026-10-10，第四轮复审 §4）

- ** 事实 **：本机绑定 `8e41d28e…`（70,518,492 B）；复审手上可用于 B 组的是
  `24e8f4c0…`（69,174,713 B）。** 两个修订版 **。
  复审实测「新增强包对可用 S2 副本在指定脚本下 ** 兼容执行 **」——
  这 ** 不能 ** 证明它与声明的那一版配对通过。** 接受这个区分。**
- ** 本方 **：** 不单方面对齐 **（擅自改成手上这一份＝替作者做发布决定，
  且会让既有独立审计的背书错配）。
- ** 给出的关闭手段 **：`checks/pin_s2.py --candidate <tarball>`——
  一致退出 0；是另一版退出 2，打印双方哈希/字节数 ** 并列出要改的三处 **
  （`sources.json`、`RELEASE_PROVENANCE.md`、重跑组装）。
- ** 状态 **：**NOT CLOSED，属作者 **；已在 `release/RELEASE_STATUS.md` 点名「谁决定、怎么关」。

## 发布-5 · ** 更正 **：门 ② 的「定义」与「调用强度」不一致（2026-10-10，第五轮复审 §5）

- ** 事由 **：组装脚本第 ② 道门的标题写「严格模式」，实际调用却 ** 没有 ** `--require-all-verified`。
  而默认严格模式 ** 容忍 `PENDING`** ⇒ 声明的强度 ≠ 实际强度。
  复审在隔离副本里把三个外部包置为 `PENDING`：原样命令仍 ** 退出 0**、只核了 2/5；
  加 `--require-all-verified` 才 ** 退出 1**。
- ** 负向实测（本方复现，只改隔离副本，不动原始归档）**：把 S2 与 v1.0-full 置为 `PENDING`：

  | 口径 | 结果 | 退出码 |
  |---|---|---|
  | 旧：默认严格模式 | 只核 **3/5** | **0** ☒ |
  | 新：`--require-all-verified` | 列出两条非 VERIFIED 并拒绝 | **1** ☒ |

  还原后 5/5 VERIFIED、退出 0。
- ** 修法 **：门 ② 加 `--require-all-verified`（** 五条必须来源任一非 VERIFIED 即失败 **）；
  加一条 `grep` 绊线防止强度被日后悄悄改弱（注释写明它是绊线、不是证明）；
  `RELEASE_STATUS.md` 与 `REPRODUCE.md` 同步写明「默认严格模式不够、
  `--available-only` 更不能代替」。
- ** 性质 **：这是 ** 发布门的强度 ** 问题，** 不是 ** 研究结果错误。

## 发布-6 · ** 更正 **：`sources.json` 的背书句指向了具体修订版（2026-10-10，第五轮复审 §5）

- ** 事由 **：`sqj-s2-release` 的 `sibling_check` 原文写「第三方独立复审解包核算过 ** 这一包 **」，
  而复审独立核到的实际 S2 是 `24e8f4c0…`，** 不是 ** 绑定的 `8e41d28e…`。
  在唯一版本确定前，这句读起来像是在为手上那一版作保。
- ** 同一类情形还有一条（复审本轮新发现）**：**v1.0 完整留档包 **——
  本机 `2380993a…`，复审环境 `215d656f…`。** 两个修订版，不是篡改。**
- ** 修法 **：两条 `sibling_check` 都改写为 ** 修订版限定 ** 的措辞并显式禁止顺延；
  加 `revision_scoped: true`；`_comment` 写明「每条背书只针对本条记录的那一个修订版」。
- ** 定版手段泛化 **：`checks/pin_s2.py` 增 `--id`（默认 `sqj-s2-release`），
  v1.0 完整包也能核（`--id simglot-v1.0-full`）。** 本方仍不替作者改 **。
- ** 状态 **：S2 与 v1.0 完整包 ** 两条都 NOT CLOSED，属作者 **，
  已列进 `release/RELEASE_STATUS.md` 的「未关闭」表。
```

Replication guide

Supplement S5 to *When Agent Evaluation Becomes Unreliable*

This guide is the map between the paper's claims and the artefacts that back them. The full package is described in `replication/README.md`; this supplement is the one-page version a reviewer can follow without reading the repository.

S5.1 Which archive produces which number

Every number in the paper comes from exactly one archive. There is no second candidate, and that is enforced: the archive hashes below are recomputed against the archives themselves by `checks/verify_freeze.py`, which exits non-zero on any mismatch.

Role	File	Bytes	SHA-256 (first 24)
`v1.0-full`	`simglot-v1.0-final.tar.gz`	81,481,922	`2380993ab6f7fafd740427f0`
`v1.0-anon`	`simglot-v1.0-anon-final.tar.gz`	80,734,508	`f0afa24478df6db74f8d688c`
`S2`	`simglot-S2-final.tar.gz`	70,518,492	`8e41d28ebfa19506a3da9232`
`enhancement`	`simglot-enhancement-final.tar.gz`	400,028	`dffa09ef0b912e3ed20f9c06`

S5.2 Claim to artefact

Every command below was checked against the archive it names; the working directory and the archive it must be run from are given in the last column. Nothing here is aspirational.

Claim in the paper	Archive	Command	Run from
Scorer defect: 4,194 vs. 2,883 database passes; 94.3% → 65.5%	v1.0	`bash scripts/offline_repro.sh`	`simglot/`
Eight checks, seven classes, no false positive on the shipped oracle	S2	`python3 oracle_audit_kit/run_audit.py`	`sqj/`
Detector ablations and the greedy minimal subset (54/54)	enhancement + S2	`bash checks/clean_room.sh` *(needs the S2 root)*	enhancement root
Component ablation, 3,240 cells	S2	`python3 swe/analyze_e1_full.py`	`sqj/`
Cross-domain reversal, −12.00 pp	S2	`python3 swe/analyze_e2_full.py`	`sqj/`
Blind adjudication, 212 items, 98.1%	enhancement	`python3 evidence/e3_inputs/build_manifest.py --check`	enhancement root

Claim in the paper	Archive	Command	Run from
AppWorld portability constraint (negative result)	enhancement	`python3 experiments/oracle_transfer/appworld_state_construction_diag.py`	enhancement root
All of the above, S2 side, in one run	S2	`bash swe/reproduce_s2.sh`	`sqj/`

The v1.0 archive unpacks to a project root; the reproduction script sits at `<root>/simglot/scripts/offline_repro.sh` and expects to be run with `<root>/simglot/` as its working directory. None of these commands needs a network connection, an API key or a model call.

S5.3 How the entry points separate computation from provenance

There are two offline entry points, one per archive side: `swe/reproduce_s2.sh` inside the S2 archive, and `checks/clean_room.sh` inside the enhancement archive. Both run their analyses in groups and print an exit code per group. Both also report the *provenance* of their inputs separately: an archive the run could not find, or could not match to its recorded hash, is reported as such and is **not** counted as verified.

That separation is deliberate and load-bearing. A run that exits 0 on the computation groups means *the computation reproduces*; it does not by itself mean every source archive was hash-matched. The packages are built so that the weaker statement cannot stand in for the stronger one: the provenance group prints an explicit status per source, and only a source whose file was found *and* whose hash equals the recorded value counts as verified. An archive that is simply not present in the run is reported as absent.

S5.4 Two things deliberately absent

- The second independently implemented benchmark used for the negative result ships its task data under a licence that forbids plaintext redistribution. The package carries the adapter, the applicability findings and the version fingerprint, but not the data, and this supplement reports no detection rate for that benchmark.
- The model-served episode transcripts.

S5.5 Environment requirements

The offline entry point needs Python 3 and the scientific stack; it needs no network access, no API key and no model client. The auditing suite itself makes zero model calls. Re-running the model-served experiments is possible in principle but is out of scope for the reviewers: the frozen per-episode records are shipped precisely so that it is not necessary.